IPSSI

2e année Bachelor Informatique

Mathématiques appliquées à l'Intelligence Artificielle

Corrections détaillées

Jour 2 — Optimisation

Auteur : Mohamed EL AFRIT

www.mohamedelafrit.com

Année académique : 2025–2026

© 2025 Mohamed EL AFRIT — IPSSI

Ce contenu est distribué sous licence

Creative Commons BY-NC-SA 4.0

https://creativecommons.org/licenses/by-nc-sa/4.0/deed.fr

Rappels — Formules utilisées

$J = \frac{1}{n}\sum(y_i - \hat{y}_i)^2$ $\quad w \leftarrow w - \alpha \cdot J'(w)$ $\quad (ax^2)' = 2ax$ $\quad ((x-a)^2)' = 2(x-a)$

$\frac{\partial J}{\partial w_1} = \frac{-2}{n}\sum x_i(y_i - \hat{y}_i)$ $\quad \frac{\partial J}{\partial w_0} = \frac{-2}{n}\sum(y_i - \hat{y}_i)$

Exercice 1

★☆☆ Fondamental

Rappel : MSE $= \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$.

a-b) Erreurs et erreurs au carré :

i	y_i	$\hat{y}_i$	$e_i = y_i - \hat{y}_i$	e_i^2
1	30	33	-3	9
2	42	40	$+2$	4
3	55	52	$+3$	9
4	38	44	-6	36
			Somme	**58**

c) MSE :

$$\text{MSE} = \frac{1}{4} \times 58 = \boxed{14{,}5\ (\text{k€})^2}$$

d) Nouveau MSE si $\hat{y}_4 = 38$: L'erreur de l'observation 4 devient $e_4 = 0$, donc $e_4^2 = 0$. Nouvelle somme : $9 + 4 + 9 + 0 = 22$.

$$\text{MSE}_{\text{nouveau}} = \frac{22}{4} = \boxed{5{,}5} \qquad \text{Diminution : } 14{,}5 - 5{,}5 = 9{,}0 \text{ soit } -62\%$$

⚠ **Erreurs fréquentes**

- Oublier le carré : sans le carré, $\sum e_i = -3 + 2 + 3 - 6 = -4$ cache la vraie ampleur des erreurs.
- Le MSE est en **(k€)**^{2}, pas en k€. L'erreur typique est $\sqrt{\text{MSE}}$.

Exercice 2

★☆☆ Fondamental

a) $f(x) = 3x^2 \implies f'(x) = \boxed{6x}$ (règle $(ax^2)' = 2ax$ avec $a = 3$).
Minimum : $6x = 0 \Rightarrow \boxed{x = 0}$.

b) $f(x) = x^2 - 6x + 9 = (x-3)^2 \implies f'(x) = \boxed{2x - 6}$
On peut dériver terme à terme : $(x^2)' = 2x$, $(-6x)' = -6$, $(9)' = 0$.

Minimum : $2x - 6 = 0 \Rightarrow \boxed{x = 3}$.

c) $f(x) = 2x + 5 \implies f'(x) = \boxed{2}$ (règle $(ax + b)' = a$). Pas de minimum (droite).

d) $f(x) = (x - 7)^2 \implies f'(x) = \boxed{2(x - 7)}$ (règle $((x-a)^2)' = 2(x-a)$).
Minimum : $2(x - 7) = 0 \Rightarrow \boxed{x = 7}$.

e) $f(x) = 5(x - 2)^2 + 3$.
Par la règle de la chaîne : $f'(x) = 5 \cdot 2(x - 2) = \boxed{10(x - 2)}$.
La constante $+3$ disparaît (dérivée d'une constante $= 0$).

À retenir

Le point où $f'(x) = 0$ est le **minimum** pour les fonctions en U (convexes). C'est le "fond de la vallée" — l'objectif de la descente de gradient.

Exercice 3

★★☆ **Fondamental**

$J(w) = (w - 4)^2$, $w_0 = 0$, $\alpha = 0{,}2$.

---

a) $J'(w) = \boxed{2(w - 4)}$

b) $J'(0) = 2(0 - 4) = \boxed{-8}$ (pente négative : la fonction descend vers la droite)

c) Mise à jour :

$$w_1 = w_0 - \alpha \cdot J'(w_0) = 0 - 0{,}2 \times (-8) = 0 + 1{,}6 = \boxed{1{,}6}$$

Le signe moins dans la formule, combiné à la dérivée négative, fait **augmenter** w — on va vers la droite, dans la bonne direction.

d) Coûts : $J(w_0) = J(0) = (0 - 4)^2 = \boxed{16}$, $J(w_1) = J(1{,}6) = (1{,}6 - 4)^2 = (2{,}4)^2 = \boxed{5{,}76}$.
Oui, le coût a diminué : $16 \to 5{,}76$ (-64%). ✓

e) Chemin parcouru : w est passé de 0 à 1,6 (sur un total de 4). Pourcentage : $1{,}6/4 = \boxed{40\%}$ du chemin en une seule itération.

Erreurs fréquentes

- Oublier le signe moins : $w_1 = w_0 - \alpha J'(w_0)$, pas $+$.
- Se tromper dans le signe de la dérivée : $2(0 - 4) = -8$, pas $+8$.

Exercice 4

★☆☆ **Fondamental**

$J(w) = w^2$, $w_0 = 5$, $\alpha = 0{,}1$.

---

a) $J'(w) = \boxed{2w}$

b) Tableau complété :

It.	w	$J'(w) = 2w$	$w_{\text{new}} = w - 0{,}1 \times 2w$	$J(w)$
0	5,000	10,0	$5 - 1{,}0 = 4{,}000$	25,00
1	4,000	8,0	$4 - 0{,}8 = 3{,}200$	16,00
2	3,200	6,4	$3{,}2 - 0{,}64 = 2{,}560$	10,24

Après 3 itérations : $w_3 = 2{,}560$.

c) Ratio : $w_3/w_0 = 2{,}560/5 = \boxed{0{,}512}$.

Le poids a parcouru $5 - 2{,}56 = 2{,}44$ vers $w^* = 0$, soit 48,8% du chemin.

d) Bonus : $w_{\text{new}} = w - \alpha \cdot 2w = w(1-2\alpha)$. Donc $w_k = w_0 \cdot (1-2\alpha)^k = 5 \times (1-0{,}2)^k = 5 \times 0{,}8^k$.

Vérification : $w_3 = 5 \times 0{,}8^3 = 5 \times 0{,}512 = 2{,}560$. ✓

À retenir

Le facteur $(1 - 2\alpha)$ doit être en valeur absolue < 1 pour converger. Donc $0 < \alpha < 1$ pour $J(w) = w^2$. Si $\alpha \geq 1$, on diverge.

Exercice 5

★★☆ Intermédiaire

3 observations, $w_1 = 3$, $w_0 = 25$.

a) Prédictions et erreurs :

i	x_i	y_i	$\hat{y}_i = 3x_i + 25$	$e_i = y_i - \hat{y}_i$	$x_i \cdot e_i$
1	2	32	$6 + 25 = 31$	$+1$	2
2	5	44	$15 + 25 = 40$	$+4$	20
3	10	55	$30 + 25 = 55$	0	0
				$\sum e_i = 5$	$\sum x_i e_i = 22$

b) Gradient par rapport à w_1 :

$$\frac{\partial J}{\partial w_1} = \frac{-2}{3} \times 22 = \boxed{-14{,}67}$$

c) Gradient par rapport à w_0 :

$$\frac{\partial J}{\partial w_0} = \frac{-2}{3} \times 5 = \boxed{-3{,}33}$$

d) Nouveaux poids ($\alpha = 0{,}01$) :

$$w_1' = 3 - 0{,}01 \times (-14{,}67) = 3 + 0{,}1467 = \boxed{3{,}147}$$
$$w_0' = 25 - 0{,}01 \times (-3{,}33) = 25 + 0{,}0333 = \boxed{25{,}033}$$

e) Les deux poids ont **augmenté**, ce qui est cohérent : les erreurs sont positives (le modèle sous-estime), donc il doit prédire **plus haut** — d'où l'augmentation de w_0 (biais) et w_1 (pente).

 CC BY-NC-SA 4.0

⚠ Erreurs fréquentes

- Oublier le $\frac{-2}{n}$ devant la somme — attention au signe !
- Confondre $\sum e_i$ et $\sum x_i e_i$: le gradient de w_1 utilise $x_i e_i$, celui de w_0 utilise e_i seul.

Exercice 6

★★☆ Intermédiaire

$J(w) = (w-3)^2$, $\alpha = 1{,}5$, *données divergentes.*

---

a) Vérification itération $0 \to 1$:
$J'(w) = 2(w-3)$. En $w_0 = 0$: $J'(0) = 2(0-3) = -6$.

$$w_1 = 0 - 1{,}5 \times (-6) = 0 + 9 = \boxed{9{,}0} \quad \checkmark$$

b) Le coût **explose** : $9 \to 36 \to 324 \to 2304 \to 9216$. Le problème : $\alpha = 1{,}5$ est **trop grand**. On saute par-dessus le minimum et on s'éloigne de plus en plus (divergence).

c) Pour $J(w) = (w-a)^2$, on a $J'(w) = 2(w-a)$ et $w_{\text{new}} = w - 2\alpha(w-a) = (1-2\alpha)w + 2\alpha a$. Convergence si $|1-2\alpha| < 1$, soit $\boxed{0 < \alpha < 1}$. Ici $\alpha = 1{,}5 > 1$: divergence.

d) Avec $\alpha = 0{,}1$:

It.	w	$J'(w) = 2(w-3)$	$J(w)$
0	0,00	−6,0	9,00
1	0,60	−4,8	5,76
2	1,08	−3,84	3,69
3	1,46	−3,07	2,36

Oui, le coût diminue régulièrement cette fois. ✓

▽ À retenir

Si le coût **augmente** au fil des itérations $\Rightarrow$ α est trop grand. Solution : **diviser α par 10** et réessayer.

Exercice 7

★★☆ Intermédiaire

$J(w) = (w-5)^2$, $w_0 = 0$, $J'(w) = 2(w-5)$.

---

a) Étudiant A ($\alpha = 0{,}05$) :

It.	w	$J'(w)$	w_{new}	$J(w)$
0	0,000	$-10{,}0$	$0 + 0{,}5 = 0{,}500$	25,00
1	0,500	$-9{,}0$	$0{,}5 + 0{,}45 = 0{,}950$	20,25
2	0,950	$-8{,}1$	$0{,}95 + 0{,}405 = 1{,}355$	16,40

$w_3 = 1{,}355$, $J(w_3) = (1{,}355 - 5)^2 = \boxed{13{,}30}$.

b) Étudiant B ($\alpha = 0{,}3$) :

It.	w	$J'(w)$	w_{new}	$J(w)$
0	0,000	$-10{,}0$	$0 + 3{,}0 = 3{,}000$	25,00
1	3,000	$-4{,}0$	$3 + 1{,}2 = 4{,}200$	4,00
2	4,200	$-1{,}6$	$4{,}2 + 0{,}48 = 4{,}680$	0,64

$w_3 = 4{,}680$, $J(w_3) = (4{,}68 - 5)^2 = \boxed{0{,}102}$.

c) Tableau comparatif :

	w_1	w_2	w_3	$J(w_3)$
$\alpha = 0{,}05$	0,500	0,950	1,355	13,30
$\alpha = 0{,}3$	3,000	4,200	4,680	0,10

d) L'étudiant B converge **130 fois plus vite** ($J = 0{,}10$ vs 13,3). Avec α encore plus grand (ex : 0,8), B risquerait d'osciller autour du minimum. La limite de convergence est $\alpha < 1$.

Exercice 8

★★☆ Intermédiaire

» Correction complète

```python
import numpy as np

# Fonction de cout et sa derivee
def J(w):
    return (w - 3) ** 2

def dJ(w):
    return 2 * (w - 3)

# Parametres
w = 0.0          # point de depart
alpha = 0.1      # taux d'apprentissage
n_iter = 20

# Boucle de descente de gradient
```

```python
for i in range(n_iter):
    grad = dJ(w)                # calculer la derivee
    w = w - alpha * grad        # mise a jour
    if i < 5 or i == n_iter - 1:
        print(f"Iter {i+1:2d} : w = {w:.4f}, J = {J(w):.4f}")

print(f"\nResultat : w = {w:.4f} (optimal : 3.0)")
print(f"Erreur : |w - 3| = {abs(w - 3):.6f}")
```

Sortie attendue :

```
Iter  1 : w = 0.6000, J = 5.7600
Iter  2 : w = 1.0800, J = 3.6864
Iter  3 : w = 1.4640, J = 2.3593
Iter  4 : w = 1.7712, J = 1.5100
Iter  5 : w = 2.0170, J = 0.9664
Iter 20 : w = 2.9885, J = 0.0001
```

Après 20 itérations, $w \approx 2{,}99$ — très proche du minimum $w^* = 3$.

Variante

Essayez $\alpha = 0{,}01$ (plus lent, 20 itérations ne suffisent pas) ou $\alpha = 0{,}5$ (très rapide, convergence en $\sim$5 itérations).

Erreurs fréquentes

- Écrire `w = w + alpha * grad` au lieu de `w - alpha * grad` — le signe moins est crucial.
- Définir `dJ` comme $2(w+3)$ au lieu de $2(w-3)$ — attention au signe dans la parenthèse.

Exercice 9

★★☆ Intermédiaire

» Correction complète

```python
import numpy as np
import matplotlib.pyplot as plt

def J(w): return (w - 3) ** 2
def dJ(w): return 2 * (w - 3)

w = -2.0
alpha = 0.15
n_iter = 25

```

```python
hist_w = [w]
hist_J = [J(w)]

for i in range(n_iter):
    w = w - alpha * dJ(w)
    hist_w.append(w)        # sauvegarder w
    hist_J.append(J(w))     # sauvegarder J(w)

# --- Graphique 1 : trajectoire ---
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5))

w_range = np.linspace(-3, 8, 200)
ax1.plot(w_range, J(w_range), 'b-', lw=2)
ax1.plot(hist_w, hist_J, 'ro-', markersize=5,
         label='Iterations')
ax1.set_xlabel('w'); ax1.set_ylabel('J(w)')
ax1.set_title('Trajectoire sur la courbe')
ax1.legend(); ax1.grid(True, alpha=0.3)

# --- Graphique 2 : convergence ---
ax2.plot(range(len(hist_J)), hist_J, 'g-o',
         markersize=4)
ax2.set_xlabel('Iteration')
ax2.set_ylabel('J(w)')
ax2.set_title('Courbe de convergence')
ax2.grid(True, alpha=0.3)

plt.tight_layout(); plt.show()
```

Description des graphiques :

— **Gauche** : points rouges "glissant" le long de la parabole bleue, de $w = -2$ vers $w = 3$.

— **Droite** : courbe verte qui décroît rapidement puis se stabilise près de 0.

⚠ Erreurs fréquentes

— Oublier d'initialiser `hist_w` et `hist_J` avec la valeur **avant** la boucle.

— Mettre `hist_J.append(J(w))` **avant** la mise à jour — on obtiendrait l'ancien coût, pas le nouveau.

Exercice 10

★★☆ Intermédiaire

» Correction complète

```python
import numpy as np
import matplotlib.pyplot as plt
```

```python

x = np.array([0,1,1,2,2,0,3,1,4,5,5,6,6,7,7,4,
    8,5,9,10,10,11,12,9,13,11,15,17,20,18], dtype=float)
y = np.array([28.0,26.5,35.0,32.0,38.0,25.0,34.5,42.0,
    38.5,44.0,41.0,48.0,43.5,50.0,39.0,47.0,51.0,40.0,
    52.0,55.0,48.5,58.0,60.0,42.0,62.0,54.0,65.0,68.0,
    72.0,58.0])
n = len(x)

w1, w0 = 0.0, 0.0
alpha = 0.001
n_iter = 5000
hist_mse = []

for it in range(n_iter):
    y_pred = w1 * x + w0              # predictions
    errors = y - y_pred                # erreurs
    dw1 = -2/n * np.sum(x * errors)    # gradient w1
    dw0 = -2/n * np.sum(errors)        # gradient w0
    w1 = w1 - alpha * dw1              # mise a jour
    w0 = w0 - alpha * dw0
    hist_mse.append(np.mean(errors ** 2))

print(f"Resultat : y = {w1:.4f} * x + {w0:.4f}")
print(f"Attendu  : y = 2.0672 * x + 31.2830")
print(f"MSE final = {hist_mse[-1]:.2f}")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5))
ax1.plot(hist_mse); ax1.set_xlabel('Iteration')
ax1.set_ylabel('MSE'); ax1.set_title('Convergence')
ax1.grid(True, alpha=0.3)

ax2.scatter(x, y, c='blue', s=30)
xl = np.linspace(0, 21, 100)
ax2.plot(xl, w1 * xl + w0, 'r-', lw=2,
         label=f'GD: y={w1:.2f}x+{w0:.2f}')
ax2.set_xlabel('Experience'); ax2.set_ylabel('Salaire')
ax2.set_title('Regression'); ax2.legend()
ax2.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()
```

Sortie attendue :

```
Resultat : y = 2.0666 * x + 31.2560
Attendu  : y = 2.0672 * x + 31.2830
MSE final = 20.46
```

Les valeurs sont très proches de la solution analytique. Avec plus d'itérations (10 000+), l'écart serait encore plus petit.

À retenir

La descente de gradient retrouve le même résultat que la formule analytique, mais de manière **itérative**. C'est plus lent ici, mais c'est la **seule** méthode possible pour les modèles non linéaires (réseaux de neurones).

Exercice 11

★★★ Avancé

» Correction complète

```python
import numpy as np
import matplotlib.pyplot as plt

def J(w): return (w - 5) ** 2
def dJ(w): return 2 * (w - 5)

alphas = [0.001, 0.01, 0.1, 0.9]
colors = ['#e74c3c', '#f39c12', '#2ecc71', '#9b59b6']
n_iter = 50
w0 = -3.0

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
w_range = np.linspace(-5, 12, 200)

print(f"{'alpha':>8} | {'w_final':>10} | {'J_final':>10}")
print("-" * 35)

for alpha, color in zip(alphas, colors):
    w = w0
    hist_w, hist_J = [w], [J(w)]
    for _ in range(n_iter):
        w = w - alpha * dJ(w)
        hist_w.append(w)
        hist_J.append(J(w))

    print(f"{alpha:>8.3f} | {w:>10.4f} | {J(w):>10.4f}")

    # Trajectoire (15 premiers points)
    ax1.plot(w_range, J(w_range), 'b-', lw=0.5, alpha=0.2)
    ax1.plot(hist_w[:15], hist_J[:15], 'o-', color=color,
             ms=4, label=f'a={alpha}')
    # Convergence
    ax2.plot(hist_J, '-', color=color, lw=2,
             label=f'a={alpha}')

ax1.set_xlabel('w'); ax1.set_ylabel('J(w)')
ax1.set_title('Trajectoire'); ax1.legend()
ax1.set_ylim(-2, 70); ax1.grid(True, alpha=0.3)
```

 CC BY-NC-SA 4.0

```python

ax2.set_xlabel('Iteration'); ax2.set_ylabel('J(w)')
ax2.set_title('Convergence'); ax2.legend()
ax2.set_ylim(-2, 70); ax2.grid(True, alpha=0.3)

plt.tight_layout(); plt.show()

# alpha=0.001 : tres lent, encore loin apres 50 iter
# alpha=0.01  : converge en ~30 iterations
# alpha=0.1   : converge en ~5 iterations (ideal)
# alpha=0.9   : oscille mais converge (limite)
```

Sortie attendue :

```
  alpha |    w_final |    J_final
-----------------------------------
  0.001 |     3.8587 |     1.3020
  0.010 |     4.9632 |     0.0014
  0.100 |     5.0000 |     0.0000
  0.900 |     5.0000 |     0.0000
```

⚠ Erreurs fréquentes

- Ne pas utiliser de boucle sur `alphas` et copier-coller le code 4 fois — c'est une mauvaise pratique.
- Oublier de réinitialiser `w = w0` au début de chaque boucle sur α.

Exercice 12

★★★ Avancé

» Correction complète

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# --- Donnees ---
x = np.array([0,1,1,2,2,0,3,1,4,5,5,6,6,7,7,4,
    8,5,9,10,10,11,12,9,13,11,15,17,20,18], dtype=float)
y = np.array([28.0,26.5,35.0,32.0,38.0,25.0,34.5,42.0,
    38.5,44.0,41.0,48.0,43.5,50.0,39.0,47.0,51.0,40.0,
    52.0,55.0,48.5,58.0,60.0,42.0,62.0,54.0,65.0,68.0,
    72.0,58.0])
n = len(x)

# --- Methode 1 : Descente de gradient ---
w1, w0 = 0.0, 0.0
```

```python
alpha = 0.001
n_iter = 10000
hist_mse = []

for _ in range(n_iter):
    err = y - (w1 * x + w0)
    w1 -= alpha * (-2/n * np.sum(x * err))
    w0 -= alpha * (-2/n * np.sum(err))
    hist_mse.append(np.mean(err ** 2))

mse_gd = hist_mse[-1]

# --- Methode 2 : scikit-learn ---
model = LinearRegression()
model.fit(x.reshape(-1, 1), y)
w1_sk = model.coef_[0]
w0_sk = model.intercept_
mse_sk = np.mean((y - model.predict(x.reshape(-1,1)))**2)

# --- Tableau comparatif ---
print("Methode            |   w1      |   w0      |   MSE")
print("-" * 52)
print(f"Gradient (10k)     | {w1:8.4f} | {w0:8.4f} | {mse_gd:.4f}")
print(f"scikit-learn       | {w1_sk:8.4f} | {w0_sk:8.4f} | {mse_sk:.4f}")

# --- Graphiques ---
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

xl = np.linspace(0, 21, 100)
ax1.scatter(x, y, c='gray', s=30, zorder=3)
ax1.plot(xl, w1*xl+w0, 'r-', lw=2,
         label=f'GD: y={w1:.2f}x+{w0:.2f}')
ax1.plot(xl, w1_sk*xl+w0_sk, 'g--', lw=2,
         label=f'sklearn: y={w1_sk:.2f}x+{w0_sk:.2f}')
ax1.set_xlabel('Experience'); ax1.set_ylabel('Salaire')
ax1.legend(); ax1.grid(True, alpha=0.3)
ax1.set_title('Droites de regression')

ax2.plot(hist_mse, 'b-', lw=1, label='GD')
ax2.axhline(mse_sk, color='green', ls='--', lw=2,
            label=f'sklearn MSE={mse_sk:.2f}')
ax2.set_xlabel('Iteration'); ax2.set_ylabel('MSE')
ax2.set_title('Convergence'); ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout(); plt.show()

# Les deux methodes trouvent quasi le meme resultat.
# Apres ~3000 iterations, GD atteint 99% de sklearn.
```

Sortie attendue :

 CC BY-NC-SA 4.0

```
Methode           |   w1     |   w0     |   MSE
-----------------------------------------------------
Gradient (10k)    |   2.0672 |  31.2819 | 20.4607
scikit-learn      |   2.0672 |  31.2830 | 20.4607
```

Les droites sont quasi identiques. La courbe de convergence montre que le MSE atteint le plateau de scikit-learn après environ 3 000 itérations.

Conclusion générale du Jour 2

La descente de gradient est un algorithme **universel** : il fonctionne pour tout modèle différentiable. Pour la régression linéaire, on obtient le même résultat qu'avec la formule analytique. Au **Jour 3**, nous l'utiliserons pour entraîner un **perceptron** — un modèle de classification où aucune formule analytique n'existe.

Variante : normaliser les features

Avec des features d'échelles différentes (ex : expérience 0–20 vs taille entreprise 15–5000), la descente de gradient converge mieux si on **normalise** les données : $x' = (x - \bar{x})/\sigma_x$. C'est une bonne pratique générale.

© 2025 Mohamed EL AFRIT — IPSSI • www.mohamedelafrit.com • CC BY-NC-SA 4.0